Agente Python para Ubuntu

https://gemini.google.com/app/84799cfea9f4ec07

User prompt: Dame un agente en python para ubuntu

Response: Aquí tienes un agente básico en Python diseñado para ejecutarse en **Ubuntu**. Este agente puede ejecutar comandos del sistema, monitorear el estado del equipo (CPU, memoria, disco) y devolver la información estructurada.

Requisitos previos

Asegúrate de tener Python 3 y la librería `psutil` instalada en tu Ubuntu:

```
sudo apt update
sudo apt install python3 python3-pip -y
pip install psutil
```

Código del Agente (`agente_ubuntu.py`)

```
import os
import platform
import subprocess
import time
import psutil

class AgenteUbuntu:
    def __init__(self, nombre="Agente-Ubuntu-01"):
        self.nombre = nombre
        self.os = platform.system()
        self.release = platform.release()

    def obtener_estado_sistema(self):
        """Monitorea el uso de hardware del equipo."""
        cpu_usage = psutil.cpu_percent(interval=1)
        ram = psutil.virtual_memory()
        disco = psutil.disk_usage('/')

        return {
            "agente": self.nombre,
            "sistema": f"{self.os} {self.release}",
            "cpu_uso_porcentaje": cpu_usage,
            "ram_usada_mb": round(ram.used / (1024 ** 2), 2),
            "ram_total_mb": round(ram.total / (1024 ** 2), 2),
            "ram_uso_porcentaje": ram.percent,
            "disco_libre_gb": round(disco.free / (1024 ** 3), 2),
            "disco_uso_porcentaje": disco.percent
        }

    def ejecutar_comando(self, comando: str):
        """Ejecuta un comando de la terminal de Ubuntu de forma segura."""
        try:
            resultado = subprocess.run(
                comando,
                shell=True,
                check=True,
                stdout=subprocess.PIPE,
                stderr=subprocess.PIPE,
                text=True
            )
            return {"exito": True, "salida": resultado.stdout.strip()}
        except subprocess.CalledProcessError as e:
            return {"exito": False, "error": e.stderr.strip()}

    def iniciar_bucle(self, intervalo=10):
        """Inicia el monitoreo continuo."""
        print(f"[{self.nombre}] Agente iniciado en Ubuntu. Presiona Ctrl+C para salir.\n")
        try:
            while True:
                estado = self.obtener_estado_sistema()
                print(f"--- Reporte {time.strftime('%H:%M:%S')} ---")
                for clave, valor in estado.items():
                    print(f"  {clave}: {valor}")
                print("-" * 30)
                time.sleep(intervalo)
```

```
        except KeyboardInterrupt:
            print("\nAgente detenido por el usuario.")

if __name__ == "__main__":
    agente = AgenteUbuntu()

    # Ejemplo 1: Ejecutar un comando de Ubuntu
    res = agente.ejecutar_comando("uptime -p")
    print(f"Tiempo encendido: {res.get('salida')}\n")

    # Ejemplo 2: Iniciar el ciclo de monitoreo cada 5 segundos
    agente.iniciar_bucle(intervalo=5)
```

Cómo ejecutarlo

1. Guarda el código anterior en un archivo llamado `agente_ubuntu.py`.
2. Ejecútalo desde la terminal:

   ```
   python3 agente_ubuntu.py
   ```

Características

- **Monitoreo nativo:** Consulta métricas de CPU, RAM y almacenamiento en tiempo real usando APIs del Kernel de Linux.
- **Ejecutor de comandos:** Puede interactuar directamente con la CLI de Ubuntu mediante subprocess.
- **Automatizable:** Puedes configurarlo como un servicio de `systemd` para que se ejecute en segundo plano al iniciar Ubuntu.

User prompt: ¿Cómo puedo integrar una API de LLM (como OpenAI) a este agente para que entienda comandos en lenguaje natural y ejecute tareas en Ubuntu?

Response: Para integrar una API de LLM (como OpenAI) a tu agente de Ubuntu, el enfoque más seguro y efectivo es darle al modelo el contexto de lo que puede hacer y usar **Function Calling** (Llamada a Funciones) o un bucle de decisiones donde el LLM traduzca la intención en un comando de terminal seguro.

Aquí tienes la versión evolucionada del agente.

1. Instalación de dependencias

Necesitas la librería oficial de OpenAI junto con `psutil`:

```
pip install openai psutil
```

Asegúrate de exportar tu clave API en la terminal de Ubuntu:

```
export OPENAI_API_KEY="tu-api-key-aqui"
```

2. Código del Agente Integrado con OpenAI (`agente_llm_ubuntu.py`)

Este script utiliza el modelo `gpt-4o-mini` para analizar la intención del usuario, generar el comando `bash` correspondiente y pedirte confirmación antes de ejecutarlo por seguridad.

```
import os
import subprocess
import psutil
from openai import OpenAI

class AgenteInteligenteUbuntu:
    def __init__(self):
        # Inicializa el cliente usando la variable de entorno OPENAI_API_KEY
        self.client = OpenAI()
        self.modelo = "gpt-4o-mini"

    def obtener_contexto_sistema(self):
        """Devuelve información básica del sistema para darle contexto al LLM."""
        cpu = psutil.cpu_percent()
        ram = psutil.virtual_memory().percent
        disco = psutil.disk_usage('/').percent
        return f"CPU: {cpu}%, RAM: {ram}%, Disco: {disco}% usado."

    def interpretar_comando(self, instruccion_usuario: str):
        """Envia la solicitud al LLM para traducir lenguaje natural a un comando Bash."""
        contexto = self.obtener_contexto_sistema()
```

```
            system_prompt = (
                "Eres un asistente experto en administración de sistemas Ubuntu Linux. "
                "El usuario te dará una orden en lenguaje natural. Tu tarea es responder "
                "ÚNICAMENTE con el comando bash exacto necesario para cumplir la solicitud. "
                "No incluyas explicaciones, ni bloques de código markdown (```bash), solo el comando. "
                "Si la solicitud es peligrosa (ej. rm -rf /), responde con la palabra 'ERROR: Comando peligroso'."
            )

            prompt_usuario = f"Estado actual del sistema: {contexto}\nOrden: {instruccion_usuario}"

            try:
                response = self.client.chat.completions.create(
                    model=self.modelo,
                    messages=[
                        {"role": "system", "content": system_prompt},
                        {"role": "user", "content": prompt_usuario}
                    ],
                    temperature=0.0 # Temperatura 0 para respuestas deterministas
                )
                return response.choices[0].message.content.strip()
            except Exception as e:
                return f"Error al conectar con OpenAI: {e}"

        def ejecutar_comando(self, comando: str):
            """Ejecuta el comando traducido en la terminal de Ubuntu."""
            print(f"\n[Ejecutando en Terminal]: {comando}")
            try:
                resultado = subprocess.run(
                    comando,
                    shell=True,
                    check=True,
                    stdout=subprocess.PIPE,
                    stderr=subprocess.PIPE,
                    text=True
                )
                return resultado.stdout if resultado.stdout else "Comando ejecutado con éxito (sin salida)."
            except subprocess.CalledProcessError as e:
                return f"Error de ejecución:\n{e.stderr}"

        def procesar_orden(self, orden: str):
            """Procesa la orden del usuario con confirmación de seguridad."""
            print(f"\nAnalizando orden: '{orden}'...")
            comando_sugerido = self.interpretar_comando(orden)

            if comando_sugerido.startswith("ERROR"):
                print(f"⚠️ {comando_sugerido}")
                return

            print(f"\n🤖 El LLM propone ejecutar: \033[1;32m{comando_sugerido}\033[0m")

            # Filtro de seguridad humano (Human-in-the-loop)
            confirmacion = input("¿Deseas ejecutar este comando en tu Ubuntu? (s/n): ").strip().lower()

            if confirmacion == 's':
                salida = self.ejecutar_comando(comando_sugerido)
                print("\n--- Resultado ---")
                print(salida)
            else:
                print("Operación cancelada por el usuario.")

    if __name__ == "__main__":
        agente = AgenteInteligenteUbuntu()

        print("=== Agente Inteligente para Ubuntu ===")
        print("Escribe 'salir' para terminar.\n")

        while True:
            orden = input("\n¿Qué quieres que haga en Ubuntu? > ")
            if orden.lower() in ['salir', 'exit']:
                break
            if orden.strip():
                agente.procesar_orden(orden)
```

Ejemplos de uso interactivo

1. **Orden:** "Muestra los 5 procesos que consumen más memoria RAM"
   - **Comando generado:** `ps aux --sort=-%mem | head -n 6`
2. **Orden:** "¿Cuánto espacio me queda en el disco?"
   - **Comando generado:** `df -h /`
3. **Orden:** "Actualiza la lista de paquetes del sistema"
   - **Comando generado:** `sudo apt update`

Consideraciones clave de arquitectura y seguridad

- **Human-in-the-Loop (Confirmación Manual):** Es fundamental pedir confirmación (`(s/n)`) antes de hacer un `subprocess.run()`. Un LLM puede alucinar o interpretar mal una orden y borrar archivos importantes.
- **Manejo de permisos:** Si el comando requiere privilegios de administrador (ej. `apt update` o reiniciar un servicio), la terminal te pedirá la contraseña de `sudo` de Ubuntu durante la ejecución.
- **Function Calling Nativo:** En proyectos de producción más grandes, en lugar de pedir texto plano al modelo, se recomienda usar la característica **Tools / Function Calling** de OpenAI para que el modelo devuelva un objeto JSON estructurado con los argumentos exactos de la función a ejecutar.
